

Section 4.1 – Illustrative Examples of Recursion

Prompt 1: What is recursion and why is it important in programming?

Answer:

Recursion occurs when a function **calls itself** to solve smaller instances of the same problem. It is important because:

- It allows complex problems to be broken into **simpler, similar subproblems**
- It is especially useful for:
 - Mathematical computations (factorial, Fibonacci)
 - Data structures (trees, graphs)
 - Algorithms (binary search, divide-and-conquer)
 - File system traversals
- It mirrors the **recursive nature of the problem**, making code more readable and easier to understand

Key principle:

- Every recursive function must have:
 1. **Base case(s)** – stop recursion
 2. **Recursive case(s)** – reduce the problem toward the base case

Prompt 2: How is the factorial function defined recursively?

Answer:

The factorial function $n!$ is defined mathematically as:

$$n! = \begin{cases} 1 & \text{if } n = 0 \\ n \cdot (n-1)! & \text{if } n \geq 1 \end{cases}$$

Explanation:

- **Base case:** $0! = 1$ stops recursion
- **Recursive case:** $n! = n * \text{factorial}(n-1)$ reduces n by 1 each call
- Example:

$$5! = 5 \cdot 4! = 5 \cdot 4 \cdot 3! = 5 \cdot 4 \cdot 3 \cdot 2! = 5 \cdot 4 \cdot 3 \cdot 2 \cdot 1! = 120$$

$$5!=5\cdot4!=5\cdot4\cdot3!=5\cdot4\cdot3\cdot2!=5\cdot4\cdot3\cdot2\cdot1!=120$$

Prompt 3: How does a recursive implementation of factorial work in Python?

Answer:

Python implementation:

```
def factorial(n):  
    if n == 0:  
        return 1  
    else:  
        return n * factorial(n-1)
```

Execution flow (factorial(4)):

1. factorial(4) → calls factorial(3)
2. factorial(3) → calls factorial(2)
3. factorial(2) → calls factorial(1)
4. factorial(1) → calls factorial(0)
5. factorial(0) → returns 1 (base case)

Return Phase (Unwinding the Stack)

factorial(1) returns

$$1 \times 1 = 1$$

factorial(2) returns

$$2 \times 1 = 2$$

factorial(3) returns

$$3 \times 2 = 6$$

factorial(4) returns

$$4 \times 6 = 24$$

Final Result

$factorial(4) = 24$

Key concepts:

- Each recursive call is **suspended** until its inner call returns
- Each call has its own **activation record** storing parameters
- Recursion stops when **base case is reached**

Prompt 4: What is a recursion trace, and why is it useful?

Answer:

A recursion trace visually represents **the sequence of recursive calls and returns**:

- Shows **which calls are active**
- Illustrates **flow of computation**
- Helps **debug recursion errors** like infinite loops

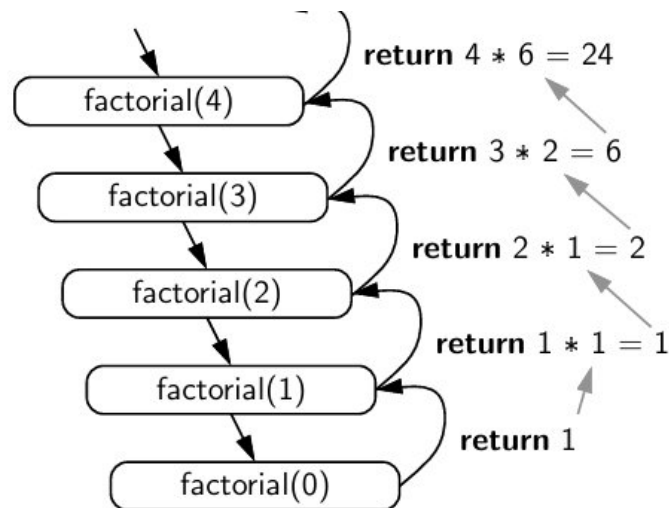


Figure 4.1: A recursion trace for the call factorial(5).

- Each indentation shows **depth of recursion**
- Returns propagate **from innermost call to outermost**

Prompt 5: What is binary search and why is it recursive?

Answer:

Binary search efficiently locates a target in a **sorted sequence**:

- Compare target to middle element
- If equal → found
- If smaller → search left half

- If larger $\rightarrow$ search right half
- Stop when interval is empty

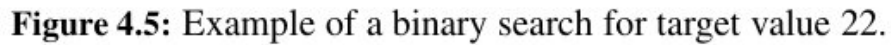
Recursive nature:

- Each recursive call **reduces the search interval by half**
- Base case: $\text{low} > \text{high} \rightarrow$ not found
- Recursive case: call binary search on **smaller interval**

Prompt 6: How is binary search implemented recursively in Python?

Answer:

```
def binary_search(data, target, low, high):  
    if low > high:  
        return False  
  
    mid = (low + high) // 2  
  
    if target == data[mid]:  
        return True  
  
    elif target < data[mid]:  
        return binary_search(data, target, low, mid - 1)  
  
    else:  
        return binary_search(data, target, mid + 1, high)
```



- Each call handles a **smaller portion** of the list
- Runs in **$O(\log n)$** time (much faster than linear search)
- Works only on **sorted sequences**

Section 4.2 – Analyzing Recursive Algorithms

Prompt 1: What does it mean to analyze a recursive algorithm?

Answer:

Analyzing a recursive algorithm means estimating **how many primitive operations** it performs as a function of the input size. The main goals are:

1. Determine **time complexity** (how long it takes)
2. Determine **space complexity** (how much memory is used)
3. Use notations like **Big-O** to summarize growth relative to input size n

Approach for recursion:

- Count operations **per function activation**
- Sum over all **recursive activations**
- Consider both **base cases** and **recursive cases**
- Recursion trace or recurrence relations help **visualize and formalize** the total work

Prompt 2: How do we analyze the factorial function recursively?

Answer:

Factorial recursion:

```
def factorial(n):  
    if n == 0:  
        return 1  
    else:  
        return n * factorial(n-1)
```

Analysis:

- Number of recursive calls: **$n + 1$** (from n down to 0)
- Work per call: **$O(1)$** (simple multiplication and comparison)
- Total work:

$$O(1) \times (n + 1) = O(n)$$

Intuition:

- Each call reduces n by 1 $\rightarrow$ linear number of calls
- Each call does constant work $\rightarrow$ linear total time

Result:

$$\text{Time complexity of factorial}(n) = O(n)$$

Prompt 3: How is binary search analyzed recursively?

Answer:

Binary search recursion:

```
def binary_search(data, target, low, high):  
    if low > high:  
        return False  
  
    mid = (low + high) // 2  
  
    if target == data[mid]:  
        return True  
  
    elif target < data[mid]:  
        return binary_search(data, target, low, mid-1)  
  
    else:  
        return binary_search(data, target, mid+1, high)
```

Analysis:

In **binary search**, each recursive call performs:

- **A constant number of primitive operations:**
 - Compute `mid`
 - Compare `target` with `A[mid]`
 - Decide left or right half

➡ Therefore, **running time depends only on how many recursive calls are made.**

❑ Size of the problem at each step

Binary search works on a **sorted sequence** using two indices:

- low
- high

The number of elements still being searched is:

$$candidates = high - low + 1$$

❑ Key observation (crucial fact)

Each recursive call **cuts the number of candidates at least in half.**

Why?

- We compute:

$$mid = \lfloor (low + high) / 2 \rfloor$$

- Then search **either**:
 - Left half: low to mid - 1
 - Right half: mid + 1 to high

In both cases, the number of remaining candidates is $\leq$ **half** of the previous size.

Mathematically:

$$Remaining\ candidates \leq \frac{high - low + 1}{2}$$

Specifically, from the definition of mid, the number of remaining candidates is either

$$(mid - 1) - low + 1 = \left\lfloor \frac{low + high}{2} \right\rfloor - low \leq \frac{high - low + 1}{2}$$

or

$$high - (mid + 1) + 1 = high - \left\lfloor \frac{low + high}{2} \right\rfloor \leq \frac{high - low + 1}{2}.$$

❑ How the problem size shrinks

Let the original sequence size be n .

Recursive call	Maximum remaining elements
0 (start)	n
1	$n / 2$
2	$n / 4$
3	$n / 8$
j	$n / 2^j$

After the **j -th recursive call**, the number of candidates is at most:

$$\frac{n}{2^j}$$

❑ When does the algorithm stop?

In the **worst case** (element not found), binary search stops when:

$$\frac{n}{2^r} < 1$$

Solve for r :

$$2^r > n$$

$$r > \log_2 n$$

Thus, the maximum number of recursive calls is:

- $r = \log n + 1$